Stay on Task: Can a small classifier correct local agents' drift?

LMA Group Project 2026, proposal. Team: lma major proj team

Member	Roll number
Shuban Biswas	2024111017
Ch Pavan Harshit	2025701057
Swayam Goyal	2024111012
Aaditya Sharma	2019113009

Domain and target user

Multi agent systems that run on a normal computer and act through tools: files, messages, social posts, bookings and trading accounts. The target user wants this private and cheap, so a big cloud model judging every step is not an option.

Problem statement

Small local agents drift and hallucinate. Partway through a task they lose track of what the user asked, make up values, claim work they never did and make risky tool calls nobody wanted. The usual fix is a generator and judge setup where an LLM judges each step, but a small model judging itself is weak [1] and a large judge is too slow and costly for every step. Tiny monitors tried so far struggled on real, unplanned mistakes [2]. We test whether a small classifier, used as is or finetuned, can serve as the judge, catching drift and hallucination at every step and steering the agents back on task.

Research question and hypothesis

Can a small classifier, used as is or finetuned, judge each step for drift and hallucination better than an LLM judge, and what does that do to task accuracy and cost?

Hypothesis	A finetuned classifier judge beats an LLM judge given the same state at catching drift and hallucination. Our full system completes more tasks than no judge and about as many as the generator and LLM judge baseline, for a fraction of its judge tokens.
Metrics	Detection: mean F1 on the two error classes rules cannot settle, off track steps and false claims, against hand labels. Tasks: share of tasks BFCL marks as passed.
Test set	Detection: 2,000 to 3,000 hand labelled steps from held out scenarios, at least 300 per headline class. Tasks: 100 held out scenarios, 4 versions each, 3 runs each.
Falsified if	Primary: the finetuned classifier does not beat the LLM judge by at least 5 F1 points with a 95% confidence interval above zero. Task claim: our full system does not beat no judge with a 95% interval above zero, or the lower end of its 95% interval against the LLM judge baseline is more than 5 points down. The two claims are tested in a fixed order: the task claim counts only if the detection claim holds, so neither needs a correction. Every other comparison is secondary and Holm corrected.

In a simulation with 300 positives and 1,700 negatives per class, judges whose errors are moderately correlated and no extra correlation within a scenario, a true gain of about 9 F1 points meets this bar about 98% of the time and a gain of about 7 points about 86% of the time. A true gain of exactly 5 points meets it only about half the time. For task accuracy, assuming independent tasks, 400 paired tasks with 3 runs each give about 80% power for a 5 point difference and about 40% or less for 2 to 3 points; clustering by scenario lowers this. Against the LLM judge we therefore claim no loss of more than 5 points at lower cost, a test two equal systems pass about 80% of the time.

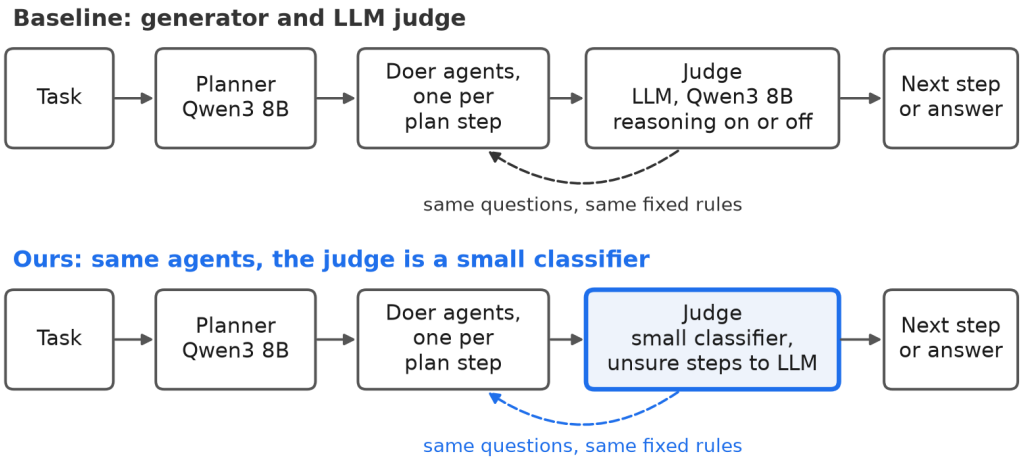
Why it matters

On a personal machine the agents touch documents, messages and accounts, so one risky tool call can delete or leak something, and every wasted step costs time.

What is missing today

1. Judging each step well needs a strong judge. Small models judge their own steps poorly [1], and even large models find it hard to point at the exact step that went wrong [3].
2. Existing step guards are 0.8B to 8B generative models focused on safety. They mostly block or flag unsafe steps rather than correct drift [4, 5, 6].
3. Small checkers cover one error type each: made up tool calls [7], claims not backed by sources [8, 9] or injected instructions [10]. Step reward models rank candidate steps for best of N picking rather than routing fixes [11, 12].
4. A public Laya head already picks actions inside a browser agent [13], and Laya's own card includes an agent trace workflow among its typed decisions [17]. Tiny monitors with repair also exist [2]. We did not find work that uses a sub 1B classifier as the judge of a multi agent pipeline, compared directly and per question with an LLM judge.

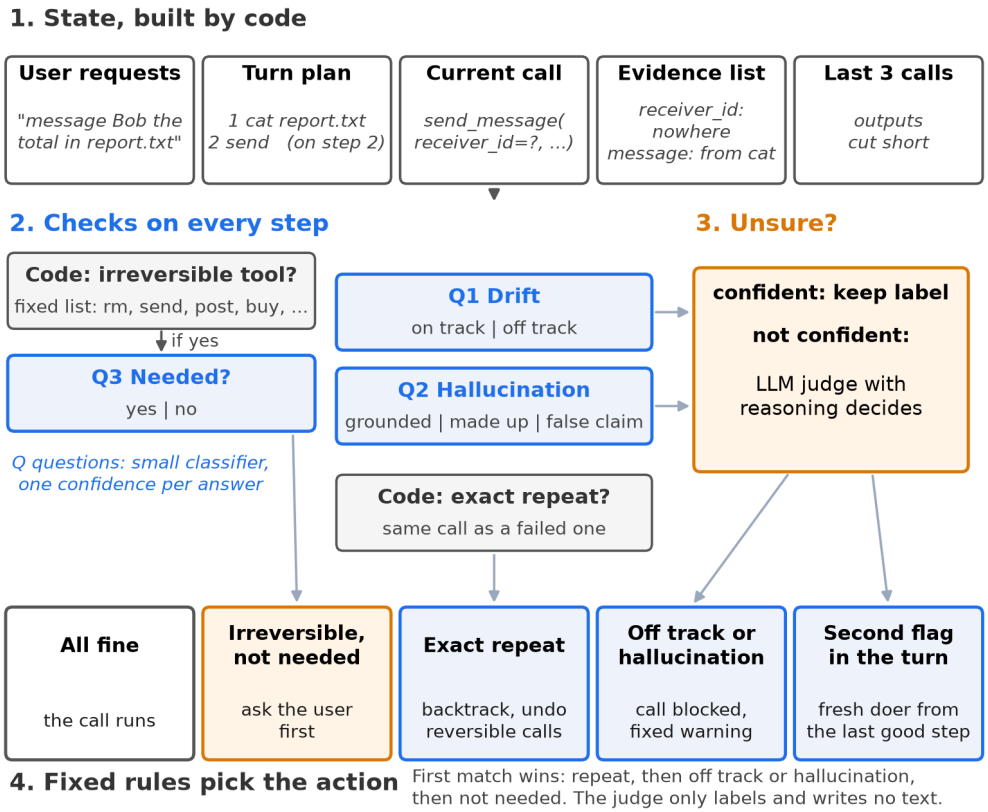
Proposed idea and architecture



The baseline is the standard generator and judge pattern [14, 15]. For each user turn a planner writes a short plan, a doer agent carries out each plan step and an LLM judge checks every step. All agents run on Qwen3 8B [16]. In ours the judge becomes Laya [17], a ModernBERT [18] typed decision model in the Jev style [19]. We use its typed decisions checkpoint (0.4B parameters, 1,024 tokens, of which about 768 hold the state) as is and fully finetuned on our data. Full finetuning a 0.4B model fits one card and takes a few hours at most, lighter than LoRA on the 8B agent would be. Our full system adds one thing: steps the classifier is unsure about go to the LLM judge.

What code does and where the real contest is. Some checks need no model, so plain code does them for every system. It checks whether a tool is irreversible (a fixed list) and whether a call exactly repeats a failed one. It also builds an evidence list that says where each value in the current call or closing message appears: the request, an earlier tool output, the tool docs or nowhere. The classifier answers the questions that need judgement. Q1 drift and Q2 hallucination are the main contest. Q3 is asked only on irreversible calls and decides whether the task needs the call, so the user is asked only about unneeded ones. On BFCL an unneeded irreversible call usually fails the turn, so Q3 positives are mostly a subset of Q1. Off track therefore takes precedence and Q3 is scored only on calls Q1 passes. If the pilot finds too few such calls, Q3 is dropped and the system asks on every irreversible call the judge passes. The full order is: exact repeat first, then off track or hallucination, then Q3. Code builds the state to fit the budget: the current and earlier user

requests, the turn plan, the current call, the evidence list and the last three calls with their outputs cut short. The LLM judge gets exactly the same state in the main comparison, so the claim is about the judge rather than what it reads. Every LLM judge condition also gets the labelling guide with a few worked examples per class. A full trace version is run as a secondary check. The judge sees each proposed call before it runs, so a flagged call is never executed.



Reasoning inside our system. Each answer comes with a confidence, calibrated on the automatic labels of the 20 validation scenarios and checked on 200 hand labelled validation steps drawn mostly from failed turns. The least sure steps for Q1 or Q2 go to the LLM judge with reasoning turned on, within a fixed budget of 10, 20 or 30% of steps, so costly reasoning is spent only on hard steps [25]. The online system uses the budget chosen on validation at Gate 2, expected around 20%. We read the reasoning on a sample of these steps to see when it fixes the classifier's label and when it breaks a right one. As an ablation, an LLM with reasoning also picks the action from the labels and confidences in place of the fixed rules. The judge itself only picks labels and never writes text. Fixed rules turn the labels into actions. All fine: the call runs. An unneeded irreversible call: ask the user. Off track or hallucination: the call is blocked and the same doer gets a fixed warning. A second flag in the same turn: a fresh doer takes over from the last good step. An exact repeat: backtrack to before the loop. The simulated environment is copied before every step, so handover and backtracking roll back reversible calls but never an irreversible one. Rolled back calls are removed from the log BFCL replays.

Capabilities used

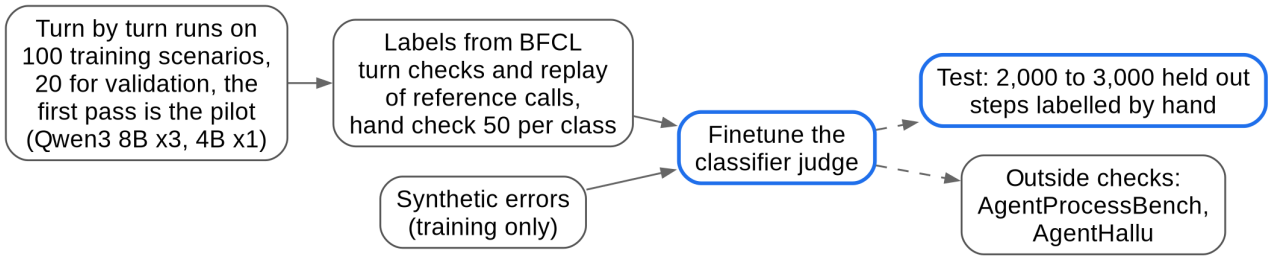
Capability	Why we need it	Metric	Ablation
Finetuning	Shows whether training on our labelled steps beats the general classifier	F1 per question and per class	As is vs finetuned, per question
Reasoning	Unsure steps go to the LLM judge with reasoning, and we analyse what that reasoning changes	F1, tokens and time per step, labels fixed vs broken by reasoning	Escalation budget 0, 10, 20, 30 and 100% of steps. LLM picks the action vs fixed rules. Question order.

Capability	Why we need it	Metric	Ablation
Agentic	Planner, doers and judge work over many turns and recover by themselves through warnings, handovers and backtracking	Task accuracy, handovers per task, correct runs broken	Correction actions: warn only, fresh doer only, all
Tool calling	Every step is a tool call or a claim about tool results, and risky calls need permission	Tool call accuracy, unapproved irreversible action rate, precision and recall of permission asks	No judge. Q3 vs asking on every irreversible call, if Q3 survives the pilot.

Expected novelty

1. A sub 1B typed decision model as the step judge of a multi agent pipeline, compared per question with an LLM judge and with plain rules.
2. Plan conditioned judging: the judge sees the turn plan, and an ablation tests whether that helps. BFCL turns are short, so we also report it on the 14% of turns that need a call and have 3 or more reference calls.
3. A confidence gated cascade: the classifier handles the steps it is sure of and the reasoning LLM only the rest, giving a cost against accuracy curve.
4. Step labels from BFCL's own turn checks and replay of reference calls instead of step by step path matching, checked against hand labels and released with the labelled steps.

Dataset and labels



The data is our own agent runs on BFCL multi turn [20], with tasks on a file system, messages, social posts, tickets, travel, trading, a car and maths. A BFCL turn passes only if the environment state matches the correct one and the agent's own tool calls returned every result the correct calls return. BFCL never reads the agent's final text, so we check that ourselves. For labelled data we run turn by turn: every user turn starts from the correct state and history, so each turn has a valid reference and one failure does not spoil the rest of the task.

One written labelling guide holds the hand definitions, and the LLM judge gets the same guide. The automatic rules approximate it for training, and we report how well each rule agrees with the hand labels per class. Q1 and Q2 apply to every tool call and every closing message. An off track call is the first call after which the turn's request can no longer be met without undoing it. Later calls in that turn are marked after drift and left out of training and scoring. An off track closing message ends the turn with the request unmet without claiming success. A false claim is a closing message that asserts something the trace contradicts or does not show. Every closing message is scored for Q2, including one after an off track call, and in a closing message false claim takes precedence over made up. A flagged closing message is held back and the doer gets the same fixed warning.

Question	Automatic label
Q1 Drift	In a failed turn, after each call we try every subsequence of the turn's reference calls, order kept, as the ending (256 replays for 8 calls), which also reruns setup calls such as cd or login. Turns with more than 8 reference calls try only endings made of the reference calls the agent has not already made with the same name and arguments, in order. The first call after which no ending makes the turn pass is off track [26]. A wrong read is not off track by itself; the call that acts on it is. In turns where the correct move is to call nothing (missing function or missing parameter versions), the request is met by saying so or asking, reads before that are on track and the first state changing call or call with an invented value is off track. A closing message is on track in a passed turn. In a failed turn with no off track call, it is on track for Q1 and a false claim for Q2 if it reports success, and off track for Q1 otherwise, so every failed turn gets a label.
Q2 Hallucination	Made up: a name, path, identifier or number, in a call or a closing message, with no evidence after normalising case, spacing and number format. A number that follows from tool outputs by simple arithmetic (sum, difference, count, mean) counts as evidenced. In a call it must also differ from the reference. A call that fills in a withheld parameter is made up. Free text such as a message body is not auto labelled. False claim: a closing message in a failed turn that reports success (done, sent, deleted, booked).
Q3 Needed	An irreversible call that is part of a correct solution is needed, otherwise not needed.

Made up labels almost follow from the evidence list, so they are reported against the rule on their own and kept out of the headline score.

Class counts. The pilot measures them. The rough estimate behind the plan: BFCL multi turn averages about 3.7 user turns per base task and 4.7 per missing function or missing parameter task, and a turn has about 2.5 judged steps. One training pass on 80 scenarios is about 1,330 turns and 3,300 steps. If 20 to 30% of turns fail:

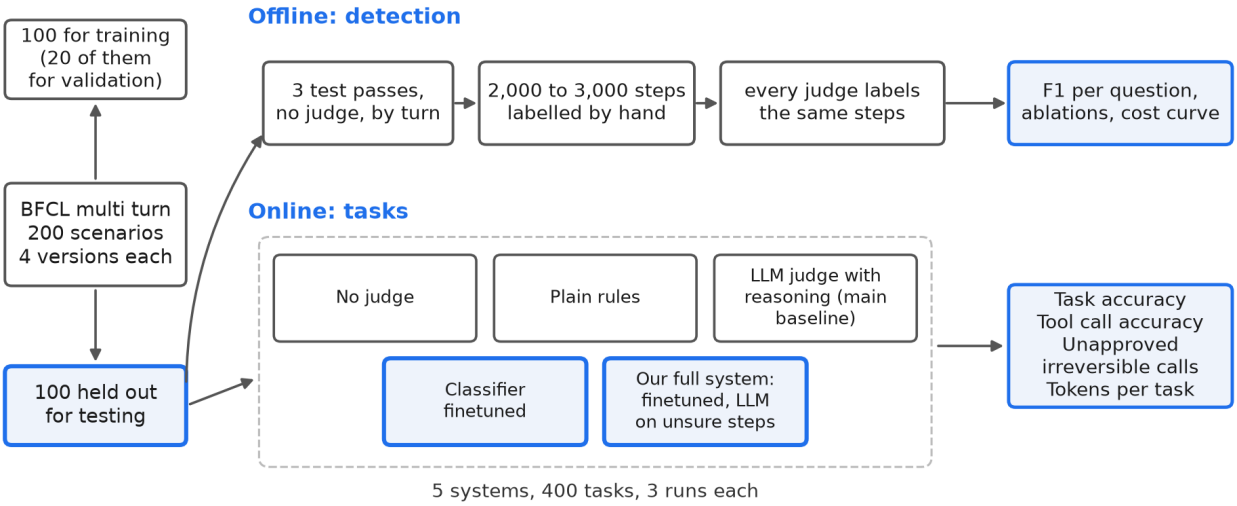
Class, per training pass	Positives	Negatives	Share
Off track step	160 to 240	about 3,100	5 to 7%
False claim	80 to 200	about 3,150	2 to 6%
Made up value	50 to 100	about 3,250	1.5 to 3%
Q3 not needed	under 30	about 120	under 20%

We therefore plan 3 passes with Qwen3 8B at temperature 0.7 and 1 pass with Qwen3 4B on the 100 training scenarios, 20 of them kept for validation, plus 3 recorded test passes. The first pass on the 20 validation scenarios is the pilot. Gate 1 needs at least 300 training examples per error class (at least 100 natural, the rest synthetic) and enough failed test turns for 300 positives per headline class. Synthetic errors are edited correct runs: an invented identifier swapped in, a call inserted or a success claim added to a failed turn. An inserted call is labelled off track only if the checker then fails the turn. Synthetic errors are never used for testing. A class weighted loss handles the rest of the imbalance.

Hand checks. Before training we hand check 50 automatic labels per class per question, about 350, and fix any rule that is right less than 80% of the time. For testing we pick failed turns at random from the recorded test passes and label every step by hand, plus 500 steps from passed turns, until off track and false claim each reach 300 positives. If 30 to 50% of failed turns end in a false claim, that is about 2,000 to 3,000 steps. Labellers see the trace and the final state but neither the automatic labels nor the BFCL verdict, and decide for themselves whether the request was met. BFCL can fail a turn whose answer is right but came through a different call, so hand labels never inherit that verdict, and we report how often labellers and BFCL agree. A fifth is labelled twice to report agreement (Cohen's kappa) and a third person settles disagreements. We also run the automatic rules on these steps and report each rule's precision and recall. About 200 validation steps are hand labelled, and 100 interventions from the online runs are checked by hand. At about a minute per step this is 13 to 18 hours per person, spread over Phases 1 and 2.

AgentProcessBench [24] and AgentHallu [3] carry human step labels and serve as outside checks, never for training. We leave out public step sets such as AgentDojo [21], ToolPRMBench [22] and Agent SafetyBench [23], since they target attacks, ranked step pairs or safety cases rather than natural drift.

Evaluation design



BFCL multi turn has 200 scenarios with 4 versions each. We split by scenario, so no version of a test scenario is seen in training, and keep 20 of the 100 training scenarios for validation. Detection is scored offline on the turn by turn test runs: every judge labels the same hand labelled steps, so any difference comes from the judge alone. Task accuracy is scored online on whole tasks, with each system running the full test set and BFCL's checker deciding success. The classifier as is is scored offline only. It was trained for other typed decisions, so it serves as a reference point rather than a contender. When a system asks for permission, the scripted user always approves, so asking cannot change task accuracy. The harness therefore measures how well systems flag risky calls, not whether a real user would stop them: we report how many asks were about unneeded calls (precision) and how many unneeded irreversible calls were blocked or asked about (recall). All tools are simulated, so nothing touches a real account.

Data and safety plan. BFCL is released under the Apache 2.0 licence and contains no personal data. Hand labels come from team members only, and the labelled steps are released under the same licence. No real account, file or message is touched at any point. The agents never run outside the simulated tools.

Baselines

1. Main baseline: generator and LLM judge, with the judge reasoning, given the same state as the classifier.
2. Offline only: the same LLM judge without reasoning or given the full trace. Qwen3 14B in 4 bit with reasoning also runs offline as a stronger reference judge.
3. No judge at all.
4. Plain rules: the code checks, a value with no evidence counted as made up, a success word (sent, deleted, booked) with no successful call of that kind counted as a false claim, a call not named in the turn plan counted as drift and permission asked on every irreversible call.
5. Our finetuned classifier alone, without escalation, offline and online.

Ablations

Ablation	What it tells us	Cost
Per question: classifier vs rules vs LLM judge	Which checks the classifier improves and which ones rules already solve	Offline, free
Finetuning gain per question	Where training helps and where it does not	Offline, free
State parts: drop the plan, the trace or the evidence list	Whether the plan helps, overall and on turns with 3 or more reference calls	Offline, 3 extra finetunes

Ablation	What it tells us	Cost
Escalation budget sweep	How much LLM reasoning buys how much F1	Offline, free, both judges' labels are stored
Question order: Q1 answer fed into Q2 or not	Whether a drift error carries over into the hallucination answer	Offline, 1 extra finetune
Synthetic errors in training or not	Whether edited examples help on natural errors	Offline, 1 extra finetune
Correction actions: warn only, fresh doer only, all, LLM picks the action	Whether the richer set of actions raises task accuracy and whether reasoning picks actions better than rules	Online, full test set, 1 run each, descriptive with intervals

Metrics

Group	What we measure
Detection	F1 per class against hand labels, with precision and recall, by input length
Task	Share of tasks BFCL marks as passed, share of interventions after which the turn passes (by warning, handover and backtrack), correct runs that a correction broke, share of closing messages with no invented number or value
Tool calling	Tool call accuracy: of the state changing calls that ran before the first failed turn, the share after which the turn can still pass by replaying reference calls. Blocked calls never ran and are left out; rolled back calls count. Exact match to a reference call is reported beside it. Permission asks per task.
Safety	Unapproved irreversible action rate: share of tasks where an unneeded irreversible call ran without a block or an ask. Asks per task beside it. Precision of asks and recall counting a call as caught if it was blocked or asked about.
Cost	Tokens and time per step and per task, share of steps sent to the LLM, handovers and backtracks per task

The headline F1 is computed on the hand labelled set as sampled. F1 reweighted to the real mix of passed and failed turns is reported next to it. Confidence intervals come from a bootstrap that resamples scenarios, keeping each scenario's runs and steps together. Every result is scored by code or by hand labels, so no AI judge grades the answers. Decision thresholds are fixed on validation data. For the finetuned classifier and the LLM judge we read every missed drift or hallucination and every false alarm by hand (about 3 more hours per person), group them by cause and show worked examples of the hardest cases.

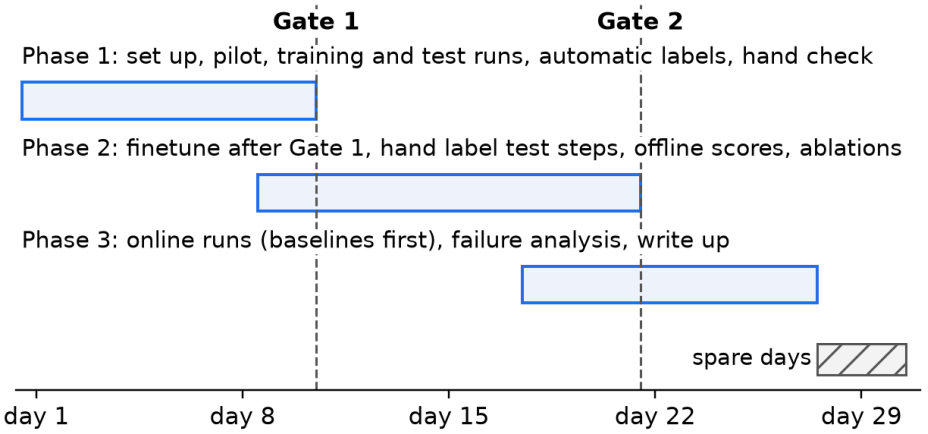
Compute plan

The Ada server gives RTX 2080 Ti cards with 11 GB, up to 4 at a time. Qwen3 8B in 4 bit and the 0.4B judge fit on one card. We serve the agents with vLLM. A task makes about 17 agent calls with 5,000 to 8,000 prompt tokens each, mostly the same tool docs, which vLLM caches and reuses. It writes about 2,000 output tokens, and the reasoning judge adds about 500 per step, capped at 1,024 thinking tokens. With little memory left for the cache only a few long prompts run at once, and the hours below assume that. The pilot measures real token counts, and the table is redone at Gate 1.

Job	GPU hours
Setup and 20 scenario pilot	4
Training and validation runs, turn by turn (Qwen3 8B 3 passes, 4B 1 pass)	15
3 test passes recorded, all judges label them and the validation steps offline, 14B reference judge	19
Finetune the classifier judge (main, 5 ablation variants, seeds)	24
Online runs: LLM judge baseline 40, our full system 16, three other systems 11 each	89
Correction action ablation, 3 new variants	16
Total	about 170, 215 with room for reruns

On 4 cards this is about 2 days of running. We also use our own machines. No paid API.

Timeline (one month)



Gate 1: class counts and label checks pass, compute redone from measured tokens.
Gate 2: offline scores done, calibration and escalation budget fixed on validation data.

Risks and fallback plan

Risk	Plan B
Too few off track steps or false claims	More 4B passes and synthetic errors for training, a fourth test pass
The classifier as is scores low, since it was trained for other decisions	It is a secondary arm. The finetuned version carries the main comparison.
States do not fit 768 tokens	Finetune with a longer window, since ModernBERT reads up to 8,192 tokens
Automatic labels are wrong too often	Fix the rule after the hand check, before training. Test labels are hand made anyway.
Hand labelling runs long	Label failed turns first and cut the passed turn steps from 500 to 300
Hand labellers disagree a lot	Tighten the labelling guide on the overlap set and relabel
Ada queue is busy, or vLLM is slow on the 2080 Ti (no bf16, no FlashAttention 2)	Move work to our own GPUs, or serve with llama.cpp
Online runs are slower than estimated	Cut the 3 runs per task to 2, then run the correction ablation on 50 scenarios

Beyond the core study

If time allows, we look at background OS tasks where a cheap judge could matter most: filing documents, mail triage, calendar upkeep and routine file clean up on a personal machine. A sub 1B judge is small enough to run next to an always on agent and can plug into common agent frameworks as a drop in check. Its per step labels and confidences give a readable audit trail of what the agents did and why they were stopped.

References

1. CRITICTOOL: evaluating self critique of LLMs in tool calling error scenarios, arXiv 2506.13977.
2. Real time detection and repair of LLM agent failures, arXiv 2608.02464.
3. AgentHallu: benchmarking automated hallucination attribution of LLM based agents, arXiv 2601.06818.
4. StepGuard: learning step level guardrails, arXiv 2608.24777.
5. SAFETY SENTRY: context aware human intervention via execute, ask, refuse routing, arXiv 2607.13594.
6. AgentDoG: a diagnostic guardrail framework for AI agent safety and security, arXiv 2601.18491.
7. Granite Guardian, arXiv 2412.07724.
8. MiniCheck: efficient fact checking of LLMs on grounding documents, arXiv 2404.10774.
9. Vectara HHEM 2.1, model card, huggingface.co/vectara/hallucination_evaluation_model.
10. LlamaFirewall and PromptGuard 2, arXiv 2505.03574.
11. ToolRM: outcome reward models for tool calling LLMs, arXiv 2509.11963.
12. AgentPRM: process reward models for LLM agents, arXiv 2511.08325.
13. laya browser: Laya as a step decision head for a browser agent, model card, huggingface.co/wuyuanpei/laya-browser.
14. Zheng et al., Judging LLM as a judge with MT Bench and Chatbot Arena, arXiv 2306.05685.
15. A survey of frontiers in LLM reasoning: inference scaling, learning to reason and agentic systems, arXiv 2504.09037.
16. Qwen3 Technical Report, arXiv 2505.09388.
17. Laya typed decision model and its typed decisions checkpoint, model card, huggingface.co/convaiinnovations/laya.
18. ModernBERT: a modern bidirectional encoder, arXiv 2412.13663.
19. TypeSafe, Jev and System One models, docs.typesafe.ai.
20. Patil et al., The Berkeley Function Calling Leaderboard (BFCL), ICML 2025.
21. AgentDojo, arXiv 2406.13352.
22. ToolPRMBench: process reward models for tool using agents, arXiv 2601.12294.
23. Agent SafetyBench: evaluating the safety of LLM agents, arXiv 2412.14470.
24. AgentProcessBench: diagnosing step level process quality in tool using agents, arXiv 2603.14465.
25. Chen et al., FrugalGPT: how to use large language models while reducing cost and improving performance, arXiv 2305.05176.
26. Wang et al., Math Shepherd: verify and reinforce LLMs step by step without human annotations, arXiv 2312.08935.